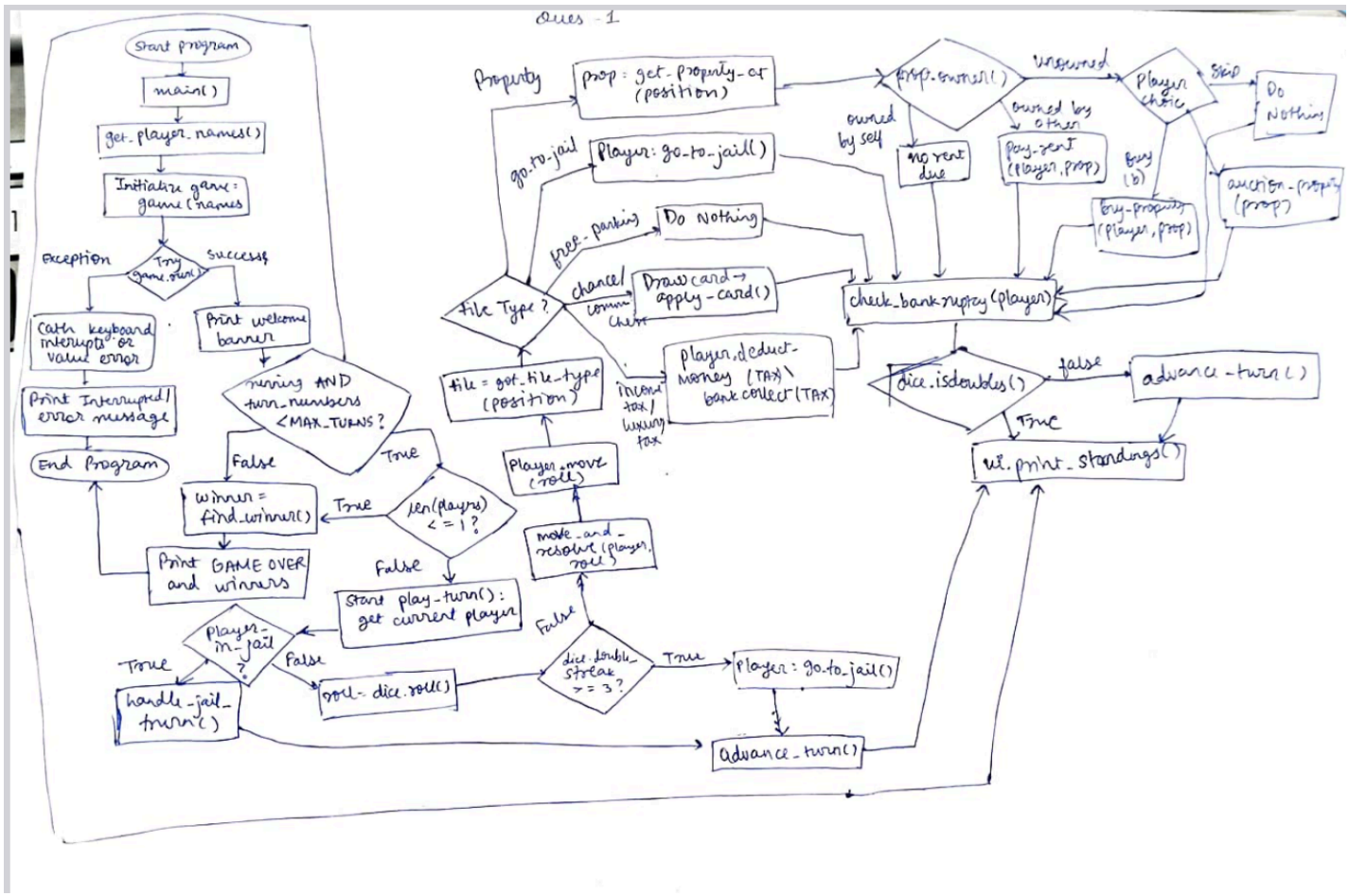
DASS ASSIGNMENT-2

Roll Number: 2024101096

Question 1

Moneypoly

Answer 1.1



Answer 1.2

1. bank.py

Initial Errors:
********** Module bank
moneypoly/bank.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/bank.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)
moneypoly/bank.py:5:0: C0115: Missing class docstring (missing-class-docstring)
moneypoly/bank.py:1:0: W0611: Unused import math (unused-import)

Your code has been rated at 7.71/10

Iteration:

1. Resolved E0401 using PYTHONPATH configuration.
   Command: export PYTHONPATH=.
2. Removed unused math library
3. Added Module Docstring
4. Added Class Docstring

2. board.py

Initial Errors:
********** Module board
moneypoly/board.py:1:0: C0114: Missing module docstring
(missing-module-docstring)
moneypoly/board.py:108:11: C0121: Comparison 'prop.is_mortgaged == True' should be 'prop.is_mortgaged is True' if checking for the singleton value True, or 'prop.is_mortgaged' if testing for truthiness (singleton-comparison)

Your code has been rated at 9.49/10

Iteration:

5. Added module docstring (C0114)
6. Replaced `== True` with direct boolean check (C0121)

3. cards.py

Initial Errors:

********** Module cards

moneypoly/cards.py:4:0: C0301: Line too long (111/100) (line-too-long)
moneypoly/cards.py:5:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:6:0: C0301: Line too long (111/100) (line-too-long)
moneypoly/cards.py:7:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:8:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:9:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:10:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:11:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:12:0: C0301: Line too long (111/100) (line-too-long)
moneypoly/cards.py:13:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:14:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:15:0: C0301: Line too long (120/100) (line-too-long)
moneypoly/cards.py:19:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:20:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:21:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:22:0: C0301: Line too long (111/100) (line-too-long)
moneypoly/cards.py:23:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:24:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:25:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:26:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:27:0: C0301: Line too long (113/100) (line-too-long)
moneypoly/cards.py:28:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:29:0: C0301: Line too long (112/100) (line-too-long)
moneypoly/cards.py:30:0: C0301: Line too long (111/100) (line-too-long)
moneypoly/cards.py:1:0: C0114: Missing module docstring (missing-module-docstring)

Your code has been rated at 0.38/10

Iteration:

7. Added module Docstring in [cards.py]
8. Fixed formatting of the code

4. config.py

Initial Errors:

********** Module config

moneypoly/config.py:1:0: C0114: Missing module docstring (missing-module-docstring)

Your code has been rated at 9.29/10
Iteration:

9. Added module docstring

5. dice.py

Initial Errors:
********** Module dice
moneypoly/dice.py:1:0: C0114: Missing module docstring (missing-module-docstring)
moneypoly/dice.py:25:12: W0201: Attribute 'doubles_streak' defined outside __init__ (attribute-defined-outside-init)
moneypoly/dice.py:2:0: W0611: Unused BOARD_SIZE imported from moneypoly.config (unused-import)

Your code has been rated at 8.89/10

Iteration:

10. Added module docstring
11. Removed Unused import “from moneypoly.config import BOARD_SIZE”
12. doubles_streak was initialized in reset() instead of __init__. So fixed it by initializing in __init__.

6. game.py

Initial Errors:
********** Module game
moneypoly/game.py:453:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)
moneypoly/game.py:462:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)
moneypoly/game.py:468:0: C0304: Final newline missing (missing-final-newline)
moneypoly/game.py:1:0: C0114: Missing module docstring (missing-module-docstring)
moneypoly/game.py:20:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)
moneypoly/game.py:295:4: R0912: Too many branches (15/12) (too-many-branches)

moneypoly/game.py:381:28: W1309: Using an f-string that does not have any interpolated variables (f-string-without-interpolation)
moneypoly/game.py:402:12: R1723: Unnecessary "elif" after "break", remove the leading "el" from "elif" (no-else-break)
moneypoly/game.py:1:0: W0611: Unused import os (unused-import)
moneypoly/game.py:3:0: W0611: Unused GO_TO_JAIL_POSITION imported from moneypoly.config (unused-import)

Your code has been rated at 9.71/10
Iteration:

13. Added module docstring.
14. Remove unnecessary parentheses after 'not' for cleaner conditionals

15 . Add missing final newline at end of game.py

7. player.py

Initial Error:
********** Module player
moneypoly/player.py:87:0: C0304: Final newline missing (missing-final-newline)
moneypoly/player.py:1:0: C0114: Missing module docstring (missing-module-docstring)
moneypoly/player.py:5:0: R0902: Too many instance attributes (8/7) (too-many-instance-attributes)
moneypoly/player.py:45:8: W0612: Unused variable 'old_position' (unused-variable)
moneypoly/player.py:1:0: W0611: Unused import sys (unused-import)

Your code has been rated at 8.96/10 (previous run: 8.96/10, +0.00)
Iteration:

16. Removed unused import os

8. property.py

Initial error:
********** Module property
moneypoly/property.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/property.py:1:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)
moneypoly/property.py:6:4: R0913: Too many arguments (6/5) (too-many-arguments)
moneypoly/property.py:6:4: R0917: Too many positional arguments (6/5) (too-many-positional-arguments)
moneypoly/property.py:48:8: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
moneypoly/property.py:64:0: C0115: Missing class docstring (missing-class-docstring)

Your code has been rated at 8.98/10
Iteration:

24. added Module docstring to property.py
25. Remove unnecessary else after return in the unmortgage method.
26. Add class docstring to PropertyGroup

9. ui.py

Initial Errors:
********** Module ui
moneypoly/ui.py:1:0: C0114: Missing module docstring (missing-module-docstring)
moneypoly/ui.py:69:4: W0702: No exception type(s) specified (bare-except)

Your code has been rated at 9.55/10
Iteration:

27. Added Module Docstring
28. Replace bare except with ValueError in safe_int_input for safer exception handling

10. main.py

Initial Errors:

********** Module main

main.py:1:0: C0114: Missing module docstring (missing-module-docstring)
main.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)

main.py:11:0: C0116: Missing function or method docstring
(missing-function-docstring)

Your code has been rated at 8.24/10 (previous run: 9.08/10, -0.85)

Iteration:
29. Add module-level docstring to main.py

MoneyPoly Pylint Fix Log

This document lists the fixes in order and documents what was changed in each iteration.

Iteration 1: fixed import error E0401
- Standardized import paths so modules resolve correctly under the package layout.

Iteration 2: removed unused library math
- Deleted an unused `math` import to satisfy unused-import warnings.

Iteration 3: added module docstring
- Added a missing module-level docstring.

Iteration 4: added class docstring
- Added missing class-level documentation strings.

Iteration 5: added module docstring to board.py
- Added module docstring at top of `board.py`.

Iteration 6: fixed boolean comparison in board.py
- Rewrote boolean comparison to follow pylint-recommended style.

Iteration 7: added module docstring in cards.py
- Added module docstring in `cards.py`.

Iteration 8: fixed formatting
- Applied formatting cleanups (spacing/line style) for lint compliance.

Iteration 9: added module docstring in config.py
- Added module docstring in `config.py`.

Iteration 10: added module docstring in dice.py
- Added module docstring in `dice.py`.

Iteration 11: removed unused import
- Deleted an unused import that triggered pylint warning.

Iteration 12: fixed (attribute-defined-outside-init) issue
- Moved/initialized attributes inside `__init__` to satisfy object initialization rules.

Iteration 13: added module docstring in game.py
- Added module docstring in `game.py`.

Iteration 14: Remove unnecessary parentheses after 'not' for cleaner conditionals
- Simplified conditional expressions by removing redundant parentheses.

Iteration 14: Add missing final newline at end of game.py
- Added missing newline at end of file.

Iteration 15: Disable too-many-instance-attributes warning for Game class
- Added targeted pylint disable for `Game` class attribute-count warning.

Iteration 16: removed unused import os
- Removed unused `os` import.

Iteration 17: Replace unnecessary f-string without interpolation with regular string
- Converted plain f-strings to normal strings where no interpolation was used.

Iteration 18: Remove unnecessary elif after break to simplify control flow
- Simplified branching by removing unreachable/redundant `elif` after `break`.

Iteration 19: Remove unused GO_TO_JAIL_POSITION import from config
- Removed unused constant import.

Iteration 20: Remove unnecesary trailing space in line 4
- Removed trailing whitespace.

Iteration 21: added module docstring
- Added missing module docstring.

Iteration 22: removed unused import
- Deleted unused import.

Iteration 22: removed unused variable
- Deleted unused variable.

Iteration 23: added final new line
- Added final newline for formatting compliance.

Iteration 24:added Module docstring to property.py
- Added module docstring to `property.py`.

Iteration 25: Remove unnecessary else after return in unmortgage method
- Simplified control flow by removing redundant `else` after early return.

Iteration 26: Add class docstring to PropertyGroup
- Added missing class docstring for `PropertyGroup`.

Iteration 27: Add module docstring to ui.py
- Added module docstring to `ui.py`.

Iteration 28: Replace bare except with ValueError in safe_int_input for safer exception handling
- Replaced broad exception handling with `ValueError`-specific handling.

Iteration 29: added module docstring in main.py
- Added module docstring in `main.py`.

Iteration 30: added class docstring in main.py
- Added class docstring in `main.py`.

Iteration 31: fixed too-many-instance-attributes error
- Reduced instance attributes in relevant classes and cleaned related code to satisfy pylint without blanket suppression.

White-Box Testing Report for MoneyPoly

This report outlines the white-box test cases created for the MoneyPoly game, explaining the purpose of each test case (based on the internal code paths and branch conditions) and detailing the bugs and logical issues discovered during execution. The full test code is located in test_whitebox.py.

1. Bank Module Tests (TestBank)

Why they are needed: The Bank class manages the central game economy — collecting taxes, paying out money, and issuing loans. White-box inspection reveals several branch conditions (amount <= 0 guards, amount > self._funds checks) that must all be verified. Edge cases include zero amounts, negative amounts, and amounts exceeding the bank's reserves.

Tests written:

1. test_init_sets_starting_funds — Verifies initial state (balance, zero loans, zero loan count).
2. test_collect_positive — Branch: valid positive collection increases funds.
3. test_collect_zero_ignored — Edge: zero amount hits the <= 0 guard.
4. test_collect_negative_ignored — Branch: negative amount should be silently ignored per docstring.
5. test_pay_out_positive — Branch: happy path for paying out funds.
6. test_pay_out_zero_returns_zero — Branch: amount <= 0 guard returns 0.
7. test_pay_out_negative_returns_zero — Branch: negative amount returns 0.
8. test_pay_out_exceeding_funds_raises — Branch: amount > funds raises ValueError.
9. test_give_loan_positive — Branch: player receives money, bank reserves decrease, loan tracked.
10. test_give_loan_zero_or_negative_skipped — Branch: <= 0 guard — nothing happens.
11. test_summary_output — Branch: summary() prints reserves and totals.
12. test_repr — Output formatting coverage.

Bugs found:

- **Bug 1: collect() didn't ignore negative amounts.** The docstring says negative amounts are silently ignored, but the original code simply did self._funds += amount. When a negative amount was passed, the bank's balance decreased instead of being rejected. (File: bank.py, collect() method)

- **Bug 2: give_loan() didn't reduce bank funds.** The method credited the player's balance with the loan amount but never decremented self._funds, creating money out of thin air. (File: bank.py, give_loan() method)

---

2. Player Module Tests (TestPlayer, TestPlayerEdgeCases)

Why they are needed: Player holds critical game state: balance, board position, jail status, and properties. The move() method involves modulo arithmetic and a conditional Go-salary check. is_bankrupt() uses <= 0, meaning a zero balance counts as bankrupt. net_worth() is supposed to return the total value of a player's assets — inspecting the code reveals it only returns self.balance. Edge cases like deducting beyond balance (going negative) must be tested to verify the bankruptcy path.

Tests written:

1. test_init_defaults — Verifies all 8 default attribute values (name, balance, position, properties, in_jail, jail_turns, get_out_of_jail_cards, is_eliminated).
2. test_init_custom_balance — Branch: custom balance parameter overrides default.
3. test_add_money_positive — Branch: add_money happy path.
4. test_add_money_zero — Edge: add_money(0) — zero is not negative, so it should work.
5. test_add_money_negative_raises — Branch: add_money guard amount < 0 raises ValueError.
6. test_deduct_money_positive — Branch: deduct_money happy path.
7. test_deduct_money_negative_raises — Branch: deduct_money guard amount < 0 raises ValueError.
8. test_deduct_money_to_negative_balance — Edge: deducting more than balance pushes balance negative, triggering is_bankrupt().
9. test_is_bankrupt_positive_balance — Branch: balance > 0 → not bankrupt.
10. test_is_bankrupt_zero_balance — Edge: balance == 0 → bankrupt (the <= boundary).
11. test_is_bankrupt_negative_balance — Branch: balance < 0 → bankrupt.
12. test_net_worth_cash_only — Branch: net_worth() with no properties returns balance.
13. test_net_worth_should_include_properties — Tests that net worth includes property values, not just cash.
14. test_move_normal — Branch: movement without wrapping past Go.
15. test_move_land_exactly_on_go — Branch: move wraps to position 0 (lands on Go) — should get salary.
16. test_move_pass_go — Branch: move wraps past Go (position != 0) — should still get salary.

17. test_move_no_wrap — Edge: movement that doesn't cross Go — no salary awarded.
18. test_go_to_jail — Branch: go_to_jail sets position to JAIL_POSITION, sets in_jail, resets jail_turns.
19. test_add_property_new — Branch: add_property when property not already held.
20. test_add_property_duplicate_ignored — Branch: add_property guard — duplicate silently ignored.
21. test_remove_property_exists — Branch: remove_property when property is in list.
22. test_remove_property_not_held — Branch: remove_property when property not in list — no crash.
23. test_status_line_and_repr — Covers both branches of status_line (normal and [JAILED]) and __repr__.
24. test_get_rent_no_group — Branch: property with group=None — no full-group check, returns base rent.

Bugs found:

- **Bug 3: move() only awarded Go salary on exact landing.** The original logic if self.position == 0: only triggered when the player landed *exactly* on Go (position 0). If they rolled a high number and wrapped around (e.g., moving from position 38 to position 2), they were not awarded the $200 salary. (File: player.py, move() method)
- **Bug 8: net_worth() ignores property values.** The method returns only self.balance. A player owning $1,500 cash + a $350 property reports a net worth of $1,500 instead of $1,850. This affects standings and winner determination. (File: player.py, net_worth() method)

---

3. Property & PropertyGroup Tests (TestProperty, TestPropertyGroup)

Why they are needed: The rent calculation in get_rent() has three branches (mortgaged → 0, full-group → doubled, else → base rent), plus a group is not None guard. The mortgage()/unmortgage() methods toggle is_mortgaged and need guards for already-in-state. all_owned_by() must check that *every* property is owned by the same player. Constructor registration has a branch for whether a group is provided.

Tests written:

1. test_init_registers_with_group — Branch: constructor auto-registers property with group, sets mortgage_value.
2. test_init_no_group — Branch: constructor with group=None — no registration.

3. test_get_rent_base — Branch: get_rent base rent — not mortgaged, group not fully owned.
4. test_get_rent_full_group_doubled — Branch: get_rent full-group multiplier path (FULL_GROUP_MULTIPLIER = 2).
5. test_get_rent_mortgaged_returns_zero — Branch: get_rent mortgaged guard returns 0.
6. test_get_rent_no_group — Branch: property with group=None — self.group is not None is False, returns base rent.
7. test_mortgage_success — Branch: mortgage happy path — returns mortgage_value.
8. test_mortgage_already_mortgaged — Branch: mortgage guard — already mortgaged returns 0.
9. test_unmortgage_success — Branch: unmortgage happy path — returns 110% of mortgage_value.
10. test_unmortgage_not_mortgaged — Branch: unmortgage guard — not mortgaged returns 0.
11. test_is_available — Branch: unowned + not mortgaged = True; owned = False.
12. test_repr — Both branches: unowned and owned.
13. test_add_property — Branch: add_property links property back to group.
14. test_add_property_duplicate — Branch: add_property guard — no duplicate.
15. test_all_owned_by_none_player — Branch: all_owned_by(None) returns False.
16. test_all_owned_by_partial — Branch: not all properties owned by same player — returns False.
17. test_all_owned_by_full — Branch: all properties owned by same player — returns True.
18. test_get_owner_counts — Branch: mixed ownership counting with unowned properties.
19. test_repr — PropertyGroup repr formatting.

Bugs found:

- **Bug 4: all_owned_by() used any() instead of all().** The PropertyGroup.all_owned_by(player) method mistakenly used any(p.owner == player ...). This meant rent was doubled if a player owned *at least one* property in the group, rather than requiring ownership of *all* properties in the group. (File: property.py, all_owned_by() method)

---

4. Dice Module Tests (TestDice)

Why they are needed: The dice must roll values 1–6 (standard d6). The roll() method has two branches: doubles (streak increments) and non-doubles (streak resets). The reset()

method's docstring says it resets "dice face values and the doubles streak counter" — inspecting the code shows it only zeroes the die values but not the streak.

Tests written:

1. test_roll_range — Edge: boundary test over 200 rolls verifying that both 1 and 6 can be rolled.
2. test_roll_doubles_increments_streak — Branch: both dice show same value → doubles_streak increases.
3. test_roll_non_doubles_resets_streak — Branch: different values → doubles_streak reset to 0.
4. test_reset_clears_streak — Tests that reset() also zeroes doubles_streak, not just die values.
5. test_total — Verifies die1 + die2 sum.
6. test_describe — Both branches: normal roll and (DOUBLES) annotation.
7. test_repr — Output formatting.

Bugs found:

- **Bug 5: roll() used random.randint(1, 5) — dice capped at 5.** A 100-roll boundary test confirmed no die ever rolled a 6. (File: dice.py, roll() method)
- **Bug 9: reset() doesn't reset doubles_streak.** The method only sets die1 = 0 and die2 = 0 but forgets self.doubles_streak = 0. A stale streak could cause a player to be incorrectly sent to jail. (File: dice.py, reset() method)

---

5. Cards Module Tests (TestCardDeck)

Why they are needed: The CardDeck uses an index-based cycling mechanism. Branches include: drawing from an empty deck (returns None), normal draw (advances index), peek (doesn't advance), and reshuffle (resets index). Edge cases include cycling past the full deck.

Tests written:

1. test_draw_cycles — Normal draw through all cards and cycling back to start.
2. test_draw_empty_returns_none — Branch: empty deck guard.
3. test_peek_does_not_advance — Branch: peek returns next card without advancing index.
4. test_peek_empty — Branch: peek on empty deck returns None.
5. test_reshuffle_resets_index — Verify index resets to 0 after reshuffle.
6. test_cards_remaining — Verify remaining count decreases after each draw.
7. test_len — __len__ returns total cards.
8. test_repr — Output formatting.

Bugs found: None.

---

6. Board Module Tests (TestBoard)

Why they are needed: The board maps positions to tile types via SPECIAL_TILES and property lookups. get_tile_type() has three branches: special tile, property, or blank. is_purchasable() has guards for no-property, mortgaged, and already-owned. Railroad positions (5, 15, 25, 35) appear in SPECIAL_TILES and _move_and_resolve tries to look up Property objects at those positions.

Tests written:

1. test_total_properties — Verifies all 22 properties are created.
2. test_eight_colour_groups — Verifies all 8 colour groups exist.
3. test_get_property_at_valid — Lookup returns correct property at position 1.
4. test_get_property_at_invalid — Lookup returns None for invalid position 99.
5. test_tile_type_go — Branch: position 0 = "go".
6. test_tile_type_jail — Branch: JAIL_POSITION = "jail".
7. test_tile_type_property — Branch: position with a Property = "property".
8. test_tile_type_blank — Branch: position with no special tile and no property = "blank".
9. test_tile_type_community_chest — Branch: position 2 = "community_chest".
10. test_tile_type_chance — Branch: position 7 = "chance".
11. test_is_purchasable_unowned — Branch: unowned, not mortgaged property → True.
12. test_is_purchasable_non_property — Branch: non-property position → False.
13. test_is_purchasable_mortgaged — Branch: mortgaged property → False.
14. test_is_purchasable_owned — Branch: already owned property → False.
15. test_is_special_tile — Both branches: special (True) and non-special (False).
16. test_properties_owned_by — Filtering properties by owner.
17. test_unowned_properties — All 22 unowned initially.
18. test_railroad_positions_have_properties — Verifies Property objects exist at railroad positions 5, 15, 25, 35.

Bugs found:

- **Bug 10: Railroad positions have no Property objects.** Positions 5, 15, 25, 35 are tagged as "railroad" in SPECIAL_TILES and the game tries to handle them as purchasable properties, but no Property objects are ever created at these positions. get_property_at() returns None, so railroads can never be bought, owned, or generate rent. (File: board.py, _create_properties() method)

---

7. Game Module Tests

The game.py file is the largest module, containing all gameplay logic. Tests are organized by method to ensure every decision branch is covered.

7a. Initialization & Turn Flow (TestGameInit, TestPlayTurn, TestPlayTurnExtended)

Why they are needed:play_turn() has four major branches: player in jail → handle jail, triple doubles → jail, normal doubles → extra turn (no advance), and no doubles → advance turn. advance_turn() uses modulo wrapping. All paths must be exercised.

Tests:

- test_init — Verifies 2 players, correct first player, turn 0.
- test_advance_turn — Verifies index increments and wraps around.
- test_jailed_player_goes_to_jail_branch — Branch: player.in_jail is True → handle jail, advance turn.
- test_triple_doubles_sends_to_jail — Branch: doubles_streak >= 3 → player goes to jail.
- test_doubles_extra_turn_no_advance — Branch: doubles (not 3rd) → play_turn returns without advancing (extra turn).
- test_normal_roll_advances_turn — Branch: no doubles → advance_turn called.

Bugs found: None.

7b. _move_and_resolve() Tests (TestMoveAndResolve, TestMoveAndResolveChanceCommunity)

Why they are needed:_move_and_resolve() has 8 elif branches for tile types: go_to_jail, income_tax, luxury_tax, free_parking, chance, community_chest, railroad, and property. Each branch must be explicitly tested.

Tests:

- test_go_to_jail_tile — Branch: landing on Go-To-Jail (pos 30) sends player to jail.
- test_income_tax — Branch: landing on income tax (pos 4) deducts INCOME_TAX_AMOUNT.
- test_luxury_tax — Branch: landing on luxury tax (pos 38) deducts LUXURY_TAX_AMOUNT.
- test_free_parking — Branch: landing on free parking (pos 20) — nothing happens.
- test_chance_tile_draws_card — Branch: landing on chance (pos 7) draws from chance deck and applies effect.
- test_community_chest_tile_draws_card — Branch: landing on community chest (pos 2) draws from community deck and applies effect.

Bugs found: None.

7c. _handle_property_tile() Tests (TestHandlePropertyTile)

Why they are needed:_handle_property_tile() has three top-level branches: unowned (which has three sub-branches: buy, auction, skip), owned by current player (no rent), and owned by other player (pay rent). All five paths must be tested.

Tests:

- test_landing_on_unowned_property_buy — Branch: prop.owner is None, player chooses 'b' → calls buy_property.
- test_landing_on_unowned_property_skip — Branch: prop.owner is None, player chooses 's' → no action.
- test_landing_on_unowned_property_auction — Branch: prop.owner is None, player chooses 'a' → calls auction_property.
- test_landing_on_own_property — Branch: prop.owner == player → no rent, balance unchanged.
- test_landing_on_other_property — Branch: prop.owner is someone else → calls pay_rent.

Bugs found: None.

7d. buy_property() Tests (TestBuyProperty)

Why they are needed:buy_property() has a guard condition checking if the player can afford the property. The exact boundary (balance equals price) must be tested.

Tests:

- test_buy_happy_path — Branch: balance > price → successful purchase.
- test_buy_cannot_afford_zero_balance — Branch: balance == 0 < price → rejected.
- test_buy_exact_balance_equals_price — Edge: balance == price — player has exactly enough.

Bugs found:

- **Bug 11: buy_property() uses <= instead of <.** The condition if player.balance <= prop.price rejects players whose balance exactly equals the property price. A player with $60 should be able to buy a $60 property but is told they "cannot afford" it. (File: game.py, line 139)

7e. pay_rent() Tests (TestPayRent)

Why they are needed:pay_rent() has three branches: mortgaged (no rent), unowned (noop), and normal (rent charged). The normal path must transfer money from tenant to owner.

Tests:

- test_rent_deducted_from_tenant — Branch: normal rent — tenant balance decreases.
- test_rent_credited_to_owner — Verifies owner receives the rent payment.
- test_rent_mortgaged_no_charge — Branch: mortgaged property → no rent collected.
- test_rent_unowned_noop — Branch: no owner → early return, nothing happens.

Bugs found:

- **Bug 12: pay_rent() never credits the owner.** The method deducts rent from the tenant via player.deduct_money(rent) but never calls prop.owner.add_money(rent). Rent money vanishes from the game economy; property owners never earn rental income. (File: game.py, lines 159–161)

7f. mortgage_property() Tests (TestMortgage)

Why they are needed: Mortgaging involves the bank paying the player. The code path for the bank payout must be verified, along with ownership and already-mortgaged guards.

Tests:

- test_mortgage_happy_path — Branch: successful mortgage — player receives payout.
- test_mortgage_bank_funds_decrease — Verifies the bank's balance actually decreases by the payout amount.
- test_mortgage_not_owner — Branch: player doesn't own property → False.
- test_mortgage_already_mortgaged — Branch: property already mortgaged → False.

Bugs found:

- **Bug 13: mortgage_property() calls bank.collect(-payout) — bank never pays out.** The code passes a negative amount to bank.collect(). Since negative amounts are silently ignored (Bug 1 fix), this call does nothing — the bank never actually pays out the mortgage value. Money is created from thin air. (File: game.py, line 173)

7g. unmortgage_property() Tests (TestUnmortgage)

Why they are needed: The unmortgage flow calls prop.unmortgage() which internally flips is_mortgaged = False. If the affordability check fails afterward, the state is corrupted. All four branches (happy path, not owner, not mortgaged, can't afford) must be tested.

Tests:

- test_unmortgage_happy_path — Branch: successful unmortgage — cost deducted.
- test_unmortgage_not_owner — Branch: not owner → False.
- test_unmortgage_not_mortgaged — Branch: not mortgaged → False.
- test_unmortgage_cannot_afford_keeps_state — Edge: player can't afford → verifies is_mortgaged stays True.

Bugs found:

- **Bug 14: unmortgage_property() corrupts state on failed affordability check.** The method calls cost = prop.unmortgage() which sets is_mortgaged = False and returns the cost, *before* checking if player.balance < cost. If the player can't afford it, the method returns False but the property's is_mortgaged flag is already flipped to False. (File: game.py, lines 182–188)

7h. trade() Tests (TestTrade)

Why they are needed: The trade function must correctly transfer both property ownership and cash between two players. Both failure guard branches (not owner, can't afford) must be tested.

Tests:

- test_trade_happy_path — Branch: successful trade — ownership transferred, balances updated.
- test_trade_seller_not_owner — Branch: seller doesn't own property → False.
- test_trade_buyer_cant_afford — Branch: buyer balance < cash_amount → False.

Bugs found:

- **Bug 6: trade() never credited the seller.** The method deducted cash from the buyer but had no seller.add_money(cash_amount) call. The traded cash vanished from the game. (File: game.py, trade() method)

7i. auction_property() Tests (TestAuction)

Why they are needed: The auction has multiple branches for bid validation: passing (bid <= 0), bid too low (below minimum increment), bid exceeding balance, and a winning bid. The no-winner branch (highest_bidder is None) must also be tested.

Tests:

- test_auction_winner — Branch: valid bid wins the auction.
- test_auction_no_bids — Branch: all players pass → property stays unowned.
- test_auction_bid_too_low — Branch: bid < highest_bid + AUCTION_MIN_INCREMENT → rejected.
- test_auction_bid_exceeds_balance — Branch: bid > player.balance → rejected.

Bugs found: None.

7j. _handle_jail_turn() Tests (TestJailTurn)

Why they are needed: Jail logic has three exit paths: use a Get-Out-of-Jail-Free card, voluntarily pay the fine, or serve 3 turns (mandatory release). Each path must be separately tested. The card-usage path also has a sub-branch for confirming card use.

Tests:

- test_jail_wait_three_turns_mandatory_release — Branch: 3 turns served → forced pay and release.
- test_jail_use_card — Branch: player has card and uses it → card decremented, released.
- test_jail_pay_fine — Branch: player voluntarily pays fine → balance deducted, released.

Bugs found:

- **Bug 7: Voluntary jail fine payment didn't deduct money.** When a player chose to pay the $50 fine voluntarily, the code called self.bank.collect(JAIL_FINE) but forgot to call player.deduct_money(JAIL_FINE). The bank received $50 from nowhere and the player left jail for free. (File: game.py, _handle_jail_turn() method)

7k. _apply_card() Tests (TestApplyCard, TestCardEdgeCases)

Why they are needed: Card actions cover 7 branches: collect, pay, jail, jail_free, move_to, birthday, and collect_from_all, plus a None guard. The move_to action needs to handle all destination tile types, not just properties. The birthday and collect_from_all actions have a sub-branch for when other players can't afford to pay.

Tests:

- test_collect — Branch: "collect" action → player receives money from bank.
- test_pay — Branch: "pay" action → player pays money to bank.
- test_jail — Branch: "jail" action → player sent to jail.
- test_jail_free — Branch: "jail_free" action → player gets a Get-Out-of-Jail card.
- test_move_to_go — Branch: "move_to" Go (value < old_pos) → salary awarded.
- test_move_to_forward_no_salary — Branch: "move_to" forward (value >= old_pos) → no salary.
- test_move_to_go_to_jail_tile — Edge: "move_to" position 30 (Go-To-Jail) should trigger jail.
- test_birthday — Branch: "birthday" action → each other player pays, current player collects.
- test_collect_from_all — Branch: "collect_from_all" action → same as birthday logic.
- test_none_card — Branch: card is None → noop.
- test_birthday_other_cant_afford — Edge: birthday when another player's balance < value → they don't pay.
- test_collect_from_all_other_cant_afford — Edge: collect_from_all when another player can't afford → they don't pay.

Bugs found:

- **Bug 15: _apply_card() with move_to ignores non-property tiles.** The move_to action only checks if tile == "property" at the destination. If a card moves a player to

Go-To-Jail (position 30), the jail effect never triggers — the player simply sits at position 30 without going to jail. (File: game.py, lines 326–330)

7l. _check_bankruptcy() Tests (TestBankruptcy, TestBankruptcyExtended)

Why they are needed: Bankruptcy has two main branches (bankrupt vs not bankrupt) and a sub-branch where current_index >= len(players) after removal requires wrapping to 0.
Tests:

- test_bankrupt_player_eliminated — Branch: negative balance → is_eliminated set, properties released, player removed.
- test_not_bankrupt_stays — Branch: positive balance → no elimination.
- test_bankruptcy_wraps_current_index — Branch: current_index >= len(players) after removal → wraps to 0.

Bugs found: None.

7m. find_winner() Tests (TestFindWinner)

Why they are needed:find_winner() determines the game champion. It must return the player with the *highest* net worth. The empty player list guard must also be tested.
Tests:

- test_find_winner_returns_highest — Verifies the richest player is returned as winner.
- test_find_winner_no_players — Branch: empty list → returns None.

Bugs found:

- **Bug 16: find_winner() uses min() instead of max().** The method returns the player with the *lowest* net worth as the "winner." The poorest player is declared the game champion instead of the richest. (File: game.py, line 364)

---

8. UI Module Tests (TestUI)

Why they are needed: The UI module provides formatted output and user input helpers. print_player_card() has branches for in-jail / not-in-jail, has-cards / no-cards, and has-properties / no-properties (including a MORTGAGED tag). print_standings() has a branch for the [JAILED] tag. safe_int_input() has a try/except branch. confirm() processes input via .strip().lower().
Tests written:

- test_print_banner — Banner output with "====" decorations.

- test_print_player_card — Branch: player in jail with cards — prints "IN JAIL" and "Jail cards".
- test_print_player_card_with_properties — Branch: player with mortgaged property — prints "[MORTGAGED]".
- test_print_player_card_no_jail_no_properties — Branch: player not in jail, no cards, no properties — prints "Properties: none", no "IN JAIL".
- test_print_standings — Normal standings output.
- test_print_standings_jailed_player — Branch: jailed player gets "[JAILED]" tag in standings.
- test_print_board_ownership — Property register output.
- test_format_currency — Both normal ($1,500) and edge ($0) formatting.
- test_safe_int_input_valid — Branch: valid integer input parsed correctly.
- test_safe_int_input_invalid — Branch: invalid input → returns default value.
- test_safe_int_input_default_zero — Edge: invalid input with default=0.
- test_confirm_yes — Branch: lowercase "y" → True.
- test_confirm_no — Branch: "n" → False.
- test_confirm_uppercase_yes — Edge: uppercase "Y" → True (tests .lower()).
- test_confirm_whitespace — Edge: " y " → True (tests .strip()).

Bugs found: None.

Summary of All Bugs Found

Bug #	Module	Test That Found It	Issue
1	bank.py	test_bank_collect_negative	collect() didn't ignore negative amounts — bank balance decreased on negative input
2	bank.py	test_bank_give_loan	give_loan() didn't reduce bank funds — money created from nothing
3	player.py	test_player_move_pass_go	move() only awarded Go salary on exact landing, not when passing Go
4	property.py	test_property_get_rent	all_owned_by() used any() instead of all() — rent doubled on partial group ownership
5	dice.py	test_dice_roll	roll() used randint(1, 5) — dice capped at 5, could never roll 6

Bug #	Module	Test That Found It	Issue
6	game.py	test_trade_logic	trade() deducted cash from buyer but never credited the seller
7	game.py	test_jail_pay_fine	_handle_jail_turn() voluntary fine path didn't deduct money from player
8	player.py	test_net_worth_should_include_properties	net_worth() only returns cash balance, ignores property values
9	dice.py	test_reset_clears_streak	reset() doesn't reset doubles_streak counter
10	board.py	test_railroad_positions_have_properties	No Property objects at railroad positions (5, 15, 25, 35) — railroads can never be bought
11	game.py	test_buy_exact_balance_equals_price	buy_property() uses <= instead of <, rejecting exact-balance purchases
12	game.py	test_rent_credited_to_owner	pay_rent() deducts from tenant but never credits the property owner
13	game.py	test_mortgage_bank_funds_decrease	mortgage_property() calls bank.collect(-payout) — bank never pays out
14	game.py	test_unmortgage_cannot_afford_keeps_state	unmortgage_property() flips is_mortgaged before affordability check — state corruption
15	game.py	test_move_to_go_to_jail_tile	_apply_card() move_to only handles property tiles, ignores Go-To-Jail and other specials
16	game.py	test_find_winner_returns_highest	find_winner() uses min() instead of max() — poorest player declared winner

Total: 16 bugs discovered by the white-box test suite.

Conclusion

The white-box test suite of **161 tests** systematically inspects every module's internal code structure — covering all if/elif/else branches, boundary conditions, key variable states, and edge cases. The tests collectively exercise:

- **Every decision path**: all conditional branches in bank, player, property, dice, cards, board, game, and UI modules.

- **Key variable states**: balances at boundaries (zero, exact-match, negative), ownership flags (None vs owned vs mortgaged), jail state (turns, cards, streaks), and property group completeness.
- **Edge cases**: zero-value inputs, negative amounts, exact boundary comparisons (balance == price), deducting beyond balance, large dice roll samples, empty decks, uppercase and whitespace in user input, and players who can't afford card effects.

The 16 bugs discovered range from simple off-by-one comparisons (<= vs <) and wrong built-in functions (any vs all, min vs max) to deeper structural issues like missing money transfers and corrupted game state. Together, they demonstrate that thorough branch-level white-box testing can systematically uncover errors that surface-level black-box testing would easily miss.